

Train a Small Language Model — Lab Session

A hands-on, from-scratch build of a tiny language model *Built on top of the Embeddings session — here we put vectors to work.*

Course repository: [tri-AI-Teaching](#) **Lab notebook (Colab):** [Lab Session Colab](#)

0. Overview

In this lab you will build a working language model from **zero** — no pretrained weights, no frameworks hiding the maths. By the end you will have a model that:

- Turns raw text into a vocabulary using **BPE** (Byte-Pair Encoding).
- Represents every word as a **vector** (embedding).
- Learns to **predict the next token** from a 4-sentence story.
- **Generates** new text one token at a time.

The whole thing runs in a Colab notebook on CPU/GPU in minutes. Everything you do here is the same recipe that powers GPT-style models — just thousands of times smaller.

Prerequisite: This lab follows the *Embeddings* session. We assume you already understand that words can be represented as vectors in "meaning space" and that similarity = closeness (cosine similarity). If you need a refresher, see the Lecture PDF.

1. One-Time Setup

Open the [Session 1 Colab notebook](#) and run the first cells.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

print(torch.__version__)
```

You do **not** need a GPU for the tiny model (32-dim, 600 steps). If you scale up (Section 9), a GPU in Colab ([Runtime](#) → [Change runtime type](#) → [GPU](#)) makes training much faster.

The training text we use is a tiny story:

```
text = """
Jide was hungry so she went looking for food.
The market was closed so she went home.
She cooked a meal and ate it happily.
```

```
After eating she felt sleepy and took a nap.
"""
```

Four sentences is enough for a small model to learn obvious patterns.

2. How Do We Turn Text Into Numbers?

Computers only understand numbers. A language model needs two numeric conversions:

1. **Tokenization** — split text into a vocabulary of tokens (characters, subwords, or words).
2. **Embedding** — map each token to a vector of numbers.

We start with **characters** (simplest possible vocabulary), then upgrade to **BPE** so the model can handle real words efficiently.

3. BPE — Building a Vocabulary by Merging

3.1 The idea

Imagine a toddler learning to read. They start with letters, then notice some pairs appear together a lot and start treating them as one chunk.

BPE works the same way:

1. **Split** every word into individual characters + a word-end marker `</w>`.
 - "crop" → "c r o p </w>"
2. **Count** every adjacent pair. Find the most frequent one.
 - ("c", "a"): 2 times, ("a", "</w>"): 9 times ...
3. **Merge** the winner into a new single token everywhere.
 - "c a" → "ca" (now one unit).
4. **Repeat** until you reach your target vocabulary size.
 - 50 merges → ~80 tokens. 1000 merges → ~1000+ tokens.

Analogy: Like building with Lego. Start with single studs (letters). Merge the most-used pairs into bigger bricks. Now "cassava" might be built from "cass" + "ava" instead of 7 separate letters.

3.2 BPE in action (trace it by hand)

Corpus: "sunny rainy sunny". Watch letters merge into subwords, step by step.

Iteration	Merge	Resulting Tokens
Start	—	s u n n y </w> r a i n y </w> s u n n y </w>
1	n + y → ny	s u n ny </w> r a i ny </w> s u n ny </w>
2	s + u → su	su n ny </w> r a i ny </w> su n ny </w>
3	su + n → sun	sun ny </w> r a i ny </w> sun ny </w>

Iteration	Merge	Resulting Tokens
4	sun + ny → sunny	sunny </w> r a i ny </w> sunny </w>
5	sunny + → sunny	sunny</w> r a i ny </w> sunny</w>

Each step merges the most frequent neighbour pair. In just 5 merges, "sunny" becomes a single token — so the model treats it as one meaningful unit instead of six separate letters.

3.3 The four functions

We only need 4 small functions to build BPE from scratch:

Function	Job
<code>get_vocab()</code>	Break text into words → split into characters → add </w>
<code>get_stats()</code>	Count how often every pair of symbols appears together
<code>merge_vocab()</code>	Replace the most frequent pair with a new merged token
<code>bpe_train()</code>	Loop: count → merge → repeat until target vocab size

```
def get_vocab(corpus):
    vocab = {}
    for word in corpus.lower().split():
        tokenized = " ".join(list(word)) + " </w>"
        vocab[tokenized] = vocab.get(tokenized, 0) + 1
    return vocab

def get_stats(vocab):
    pairs = {}
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols) - 1):
            pair = (symbols[i], symbols[i + 1])
            pairs[pair] = pairs.get(pair, 0) + freq
    return pairs

def merge_vocab(pair, vocab):
    new_vocab = {}
    bigram = " ".join(pair)
    for word, freq in vocab.items():
        new_word = word.replace(bigram, "".join(pair))
        new_vocab[new_word] = freq
    return new_vocab

def bpe_train(corpus, num_merges):
    vocab = get_vocab(corpus)
    merges = []
    for _ in range(num_merges):
        pairs = get_stats(vocab)
        best = max(pairs, key=pairs.get)
```

```
vocab = merge_vocab(best, vocab)
merges.append(best)
return vocab, merges
```

Try it: Run `bpe_train("sunny rainy sunny", 5)` and check that the output matches the table above.

3.4 Switching to a production tokenizer

Our hand-rolled BPE is tiny. Real models use a pretrained tokenizer (e.g. GPT-2's). In the notebook, after building your own, swap in the production one:

```
from transformers import GPT2TokenizerFast
tokenizer = GPT2TokenizerFast.from_pretrained("gpt2")
```

This gives you a ~50,000-token vocabulary for free. The architecture we build next works with **any** tokenizer.

4. Embeddings — Words as Vectors

A number alone (a token ID) has no meaning — "397" and "401" don't tell us the words are related.

Embeddings fix this by mapping each token to a **vector of numbers** where similar words live close together.

- **Token ID → Vector:** Each token gets its own embedding — a list of, say, 32 numbers. Think of it as a *coordinate in meaning-space*.
- **Similar → Nearby:** "cat" and "dog" have similar vectors (both pets). "king" is close to "queen" but far from "apple".
- **Learned During Training:** Embeddings start as random noise. As the model trains, it pushes/pulls words closer or farther apart to capture meaning.

```
Embedding for "king" → [0.23, -1.45, 0.67, ..., 0.12] (32 numbers)
Embedding for "queen" → [0.25, -1.40, 0.62, ..., 0.15] (very similar!)
Embedding for "apple" → [-0.98, 2.10, -0.33, ..., 0.88] (very different)
```

Link to last session: You explored embeddings in depth — t-SNE maps and cosine similarity. *Today we put them to work:* those same vectors become the model's memory, learned live as it trains.

Hands-on check: Open `similarity_answers.md` in the repo and work through the integer-vector cosine examples. Compute similarity between `[6, 9, 1]` and `[2, 3, 7]`, then between two near-identical vectors, to build intuition.

5. Creating Training Data — The Sliding Window

A language model's only job: **predict the next token**. We create flashcards: "Given these tokens, what comes next?"

Context → Target

```
block_size = 8    # How many tokens the model sees at once
batch_size = 4    # How many examples we process together

def get_batch():
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i+block_size] for i in ix])    # Context
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])    # Target
    return x.to(device), y.to(device)
```

Example flashcard:

Context: "Jide was hungry so she went look"
 Target: " was hungry so she went looking for"

The model sees 8 tokens at a time and must predict token #9. We make thousands of these from our tiny story.

6. The Tiny Brain — Our SLM Architecture

A miniature version of what powers ChatGPT. Three parts:

1. **Token Embedding** — Each token ID → a vector of 32 numbers. Similar words get similar vectors.

```
nn.Embedding(vocab_size, n_embd=32)
```

2. **Position Embedding** — Each position (1st, 2nd, 3rd...) → a vector. Helps the model know word order.

```
nn.Embedding(block_size, n_embd)
```

3. **Linear Head** — Predicts which token comes next. A "what's the most likely next word?" machine.

```
nn.Linear(n_embd, vocab_size)
```

```
class TinySLM(nn.Module):
    def __init__(self, vocab_size, n_embd):
        super().__init__()
```

```

self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
self.position_embedding_table = nn.Embedding(block_size, n_embd)
self.lm_head = nn.Linear(n_embd, vocab_size)

def forward(self, idx, targets=None):
    B, T = idx.shape
    tok_emb = self.token_embedding_table(idx)
    pos_emb = self.position_embedding_table(torch.arange(T, device=device))
    x = tok_emb + pos_emb
    logits = self.lm_head(x)
    if targets is None:
        return logits, None
    B, T, C = logits.shape
    loss = F.cross_entropy(logits.view(B*T, C), targets.view(B*T))
    return logits, loss

```

The two **embedding tables** are the heart of the model — they store the meaning of every word, exactly the vectors you explored last session. Training is just the model *learning better embeddings* so it can predict what comes next.

Total: ~6.5 million parameters. Tiny by today's standards (GPT-4 has trillions), but enough to learn patterns!

7. Training — Becoming Less Wrong

A fresh model guesses randomly. Training fixes that — one tiny adjustment at a time.

1. **Forward Pass** — Feed context, get prediction, calculate loss. Loss = "how wrong are we?" (lower is better).
2. **Backward Pass** — Use `loss.backward()` to calculate the gradient — the direction to nudge each weight.
3. **Update Weights** — `optimizer.step()` applies the nudges. The model becomes slightly less wrong.

```

optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

for step in range(600):
    xb, yb = get_batch()                # Grab a batch
    logits, loss = model(xb, yb)         # Forward pass
    optimizer.zero_grad(set_to_none=True)
    loss.backward()                      # Backward pass
    optimizer.step()                     # Update weights
    if step % 100 == 0:
        print(f"Step {step}: Loss = {loss.item():.4f}")

```

Watch the loss go from **~10** (random guessing) down to **~1.5** (learning patterns).

8. Let It Speak!

After training, our tiny model can generate text. We give it a starting word and ask: "What comes next?"

```
def generate(self, idx, max_new_tokens, temperature=1.0):
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -block_size:]          # Only use last 8 tokens
        logits, _ = self(idx_cond)               # Predict
        logits = logits[:, -1, :] / temperature  # Scale by temperature
        probs = F.softmax(logits, dim=-1)         # Convert to probabilities
        idx_next = torch.multinomial(probs, 1)    # Sample
        idx = torch.cat((idx, idx_next), dim=1)  # Append
    return idx
```

Temperature trick: Lower = more focused/repetitive, higher = more creative/risky. Try `temperature=0.8`!

9. Making Our Model Smarter

Our first model was tiny (32-dim embeddings, 8-token context, 600 steps). Let's level up:

Change	From → To	Effect
<code>block_size</code>	8 → 32	Wider context = sees more of the story → more coherent text
<code>n_embd</code>	32 → 64	Richer token vectors → better pattern capture
Steps	600 → 5000	More training → more chances to learn

Result: A model that generates much more coherent text.

Before: Jide was hungry she market closed market
 After: Jide was hungry so she went looking for food

Scale the recipe further: The same architecture at a larger size reaches **12,937,781 parameters** — more capacity means more patterns it can capture. (Our BiggerSLM uses ~6.5M; bigger still = smarter still.)

Experiment: Change these 3 numbers in the notebook and watch the difference. What happens if you make `block_size` too large on a 4-sentence story?

10. What You Learned in the Lab

1. **BPE builds vocabulary by merging frequent pairs** — start with characters, merge the most common pair, repeat. The merge order reveals language patterns.
2. **Vocab size controls efficiency** — smaller vocab = more tokens per word (handles rare words); larger vocab = fewer tokens (faster, but bigger model).
3. **Sliding window creates training data** — every language model learns by predicting the next token; the sliding window turns raw text into context→target flashcards.

4. **Even tiny models can learn patterns** — with only ~6.5M parameters and a 4-sentence story, our model learned to generate coherent text. That's the magic of neural networks.

The big idea: Every word's meaning lives in its **embedding** — the same vectors you explored last session. When we train this little model, we are really just *learning better embeddings* so it can predict the next word. That's the whole magic of language models!

11. Resources

- **Original reference Colab:** [Openen Notebook](#)
- **Lab notebook:** [Session Core Colab](#)
- **Similarity worked examples:** [Open the Examples](#)
- **Quiz:** [Open the Questions](#)
- **Course slides:** [Train a Small Language Model — slides](#)
- **Embeddings lecture deck:** [embeddings.html](#)

Instructor: Kaletsidik Ayalew **GitHub:** [AlexKaIII](#) · **LinkedIn:** [kaletsidik-ayalew](#)